

Deepfake Face Image Detection Using a CPU-Only Binary MobileNetV2 Baseline on FaceForensics++

Abdelrahman Ibrahim Kamal Eldin

ID: 20596377

Dept. of Computing

Queens University

abdelrahman.ibrahim@queensu.ca

Aya Said Abdallah Noah

ID: 20596338

Dept. of Computing

Queens University

25xgvp@queensu.ca

Esraa Mohammed Elsayed

ID: 20596301

Dept. of Computing

Queens University

esraa.farag@queensu.ca

Abstract—The increasing realism of deepfake and facial manipulation techniques poses serious challenges to the integrity of digital media, enabling misinformation, identity fraud, and reputational harm at scale. This paper presents a binary deepfake detector real versus fake, built on a lightweight MobileNetV2 backbone and trained on FaceForensics++, a standard benchmark containing manipulated facial videos produced by four forgery methods: DeepFakes, Face2Face, FaceSwap, and NeuralTextures, at c23 compression. The system is designed to run entirely on CPU and targets reproducible evaluation under the official pair-based split protocol to prevent data leakage. Training employs two-stage transfer learning, WeightedRandomSampler, class-weighted cross-entropy loss with label smoothing, and validation-based threshold calibration. The current baseline achieves 64.63% frame-level accuracy, 0.604 Macro-F1, and 0.637 ROC-AUC on the test set. At group level, aggregating frame probabilities per video pair, the model reaches 59.26% accuracy and 0.667 AUC. Robustness evaluation under JPEG recompression and Gaussian noise reveals that post-processing degrades both ranking ability and per-class recall, confirming that compression robustness remains a key open challenge for practical deepfake detection.

Keywords—Deepfake detection, FaceForensics++, MobileNetV2, binary classification, transfer learning, robustness, JPEG compression, Gaussian noise

I. INTRODUCTION

Deepfakes and facial manipulation methods have become a major threat to digital media authenticity because realistic forgeries can be used for misinformation, impersonation, or reputational harm. FaceForensics++ [1] was introduced to standardize research on manipulated face detection and includes four forgery families: DeepFakes, Face2Face, FaceSwap, and NeuralTextures.

The main goal of this project is to build a practical deepfake detector that distinguishes real from fake facial images under constrained computation, running entirely on CPU. This midterm report focuses on establishing a reliable binary classification baseline before extending to multi-class manipulation attribution. The implementation prioritises protocol correctness official pair-based splits, proper label assignment, and threshold calibration as academically sound evaluation is a prerequisite for meaningful comparison in future phases.

The scope of this midterm includes: a reproducible FaceForensics++ preprocessing pipeline; the official pair-based split protocol; a MobileNetV2 binary classifier with two-stage transfer learning; frame-level and group-level evaluation; and an initial robustness assessment under JPEG compression and Gaussian noise.

II. RELATED WORK

FaceForensics++ [1] introduced one of the most widely used facial forgery benchmarks, containing original sequences together with manipulated counterparts generated by four methods under multiple compression levels. The dataset demonstrated the value of focusing on manipulated facial regions rather than full images.

MobileNetV2 [2] is a mobile-friendly convolutional architecture using depthwise separable convolutions, inverted residuals, and linear bottlenecks, making it a practical backbone for a lightweight deepfake detector on CPU-constrained hardware.

Xception [3] has become a strong baseline in image forensics and deepfake detection, providing a useful reference for comparing heavier and lighter detectors.

Face X-Ray [4] focuses on generalised forgery detection by exposing blending boundaries, motivating the importance of robustness beyond clean benchmark conditions.

Qi et al. [5] demonstrate that deepfake detectors rely heavily on high-frequency convolutional traces left by generation pipelines. This directly explains the sensitivity observed in our robustness experiments: JPEG compression and Gaussian noise destroy precisely these fine-grained artifacts, degrading Macro-F1 and AUC while leaving raw accuracy misleadingly stable.

III. METHODOLOGY

A. Dataset and Split Protocol

We use FaceForensics++ at c23 compression, which balances visual quality and file size. Manipulated sequences follow the <target> <source> naming convention, requiring pair-level rather than frame-level splitting. We use the official train.json (360 pairs), val.json (70 pairs), and test.json (70 pairs) files with no overlap across sets. Manipulated samples are assigned by exact pair membership; real samples are assigned using the original identities in the same split files. Overlap verification confirmed zero shared groups between all three splits.

B. Preprocessing

Ten frames are sampled per video at uniform intervals to ensure temporal diversity. Face regions are extracted using an OpenCV Haar cascade, with a centre-crop fallback if no face is detected. Each crop is resized to 224×224 pixels and normalised using ImageNet statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]). This concentrates the model on the face region where deepfake artifacts are most pronounced.

C. Model Architecture

The baseline is MobileNetV2 initialised with ImageNet-pretrained weights. The original classification head is replaced with: Dropout (0.3) → Linear (1280, 256) → ReLU → Dropout (0.2) → Linear (256, 2). Stage 1 freezes the entire backbone and trains only the new head. Stage 2 unfreezes the last four feature blocks and fine-tunes with a smaller learning rate to preserve general visual features while adapting to forgery cues.

D. Training and Evaluation Protocol

All experiments run on CPU only (Windows, Python 3.11, PyTorch, OpenCV, scikit-learn, NumPy, Pandas, Matplotlib). Class imbalance (fake:real ≈ 2:1) is addressed using WeightedRandomSampler with class-weighted cross-entropy loss (real weight = 4.0, fake weight = 1.0) and label smoothing = 0.05. The decision threshold is tuned on the validation set via grid search over [0.05, 0.95]. Both frame-level and group-level metrics (Accuracy, Macro-F1, ROC-AUC) are reported. Full training configuration is in Table I.

TABLE I. TRAINING CONFIGURATION

PARAMETER	STAGE 1	STAGE 2
Optimizer	AdamW	AdamW
Learning Rate	1×10^{-3}	1×10^{-5}
Batch Size	16	16
Weight Decay	1×10^{-4}	1×10^{-4}
Label Smoothing	0.05	0.05
Unfrozen Layers	Head only	Last 4 blocks
Epochs	5	15
Early Stop	-	Patience = 4

IV. PRELIMINARY EXPERIMENTS AND RESULTS

A. Training Behaviour

Stage 1 converged to 56.5% validation accuracy, 0.565 Macro-F1, and 0.719 ROC-AUC after 5 epochs. Stage 2 fine-tuning improved performance until early stopping triggered at epoch 5 (patience = 4). The best threshold for accuracy was 0.168 (val accuracy 73.67%, Macro-F1 0.675); the best for Macro-F1 was 0.192 (val accuracy 73.33%, Macro-F1 0.679). Both thresholds are well below 0.5, confirming the need for calibration under class-imbalanced training. Curves are shown in Fig. 1.

Fig. 1. Training and validation curves for the CPU-only MobileNetV2 binary baseline on FaceForensics++ using the official pair-based split.

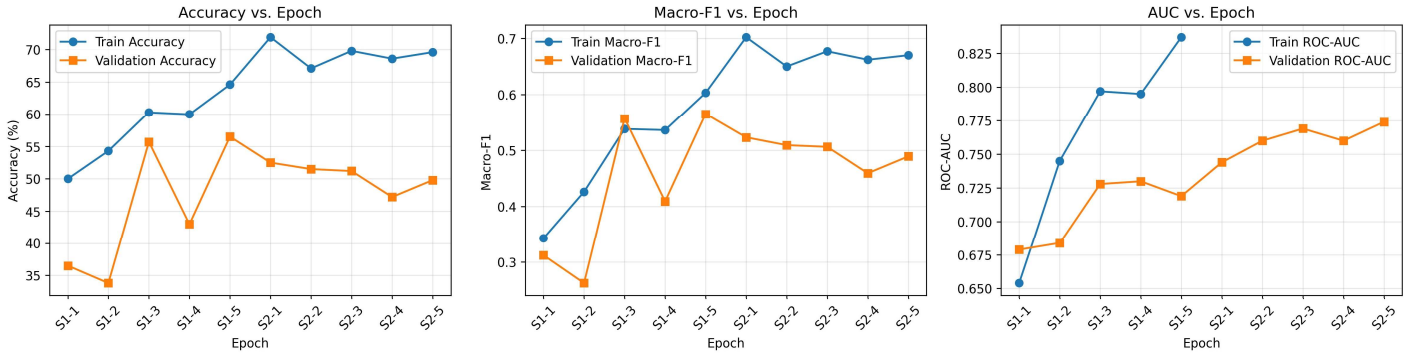


Fig. 1. Training and validation curves (Accuracy, Macro-F1, ROC-AUC) for the CPU-only MobileNetV2 binary baseline on FaceForensics++ using the official pair-based split.

B. Clean Test Results

On the clean frame-level test set, the final baseline achieved 64.63% Accuracy, 0.604 Macro-F1, and 0.637 ROC-AUC. At group level the detector achieved 59.26% test Accuracy, 0.593 Macro-F1, and 0.667 ROC-AUC. The stronger group-level validation AUC (0.850) vs frame-level (0.637) confirms that probability aggregation provides a more stable estimate on this video-based dataset. Results are in Table II; the confusion matrix is in Fig. 2.

TABLE II. CLEAN EVALUATION RESULTS

LEVEL	ACCURACY	MACRO-F1	AUC
Frame (Test)	64.63%	0.604	0.637
Group (Val)	56.67%	0.562	0.850
Group (Test)	59.26%	0.593	0.667

The confusion matrix (Fig. 2) shows 263 of 360 fake frames correctly classified (73.1% recall) but only 86 of 180 real frames (47.8% recall). This asymmetry reflects the

lowered threshold improving fake recall at the cost of real precision.

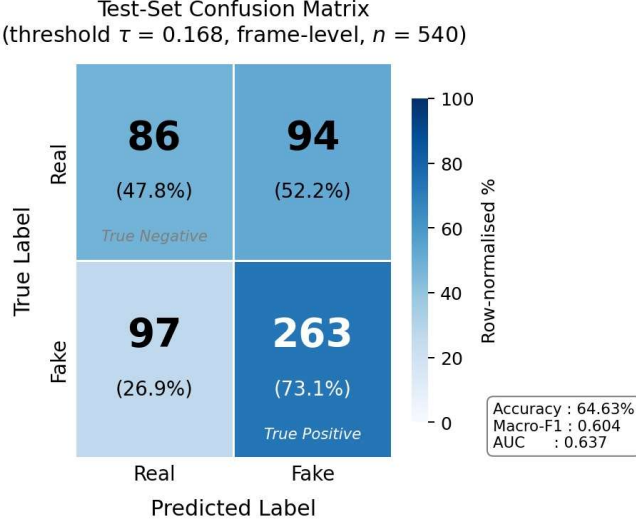


Fig. 2. Test-set confusion matrix ($\tau = 0.168$, frame-level, $n = 540$).

C. Robustness to JPEG Compression and Gaussian Noise

We evaluated the final checkpoint under JPEG recompression (quality 80, 60, 40, 20) and Gaussian noise (std = 5, 10, 20, 40 on pixel scale 0–255). Results are in Table III.

JPEG consistently degrades performance; the largest accuracy drop occurs at $q=40$ (49.81%), with a partial recovery at $q=20$ (55.93%), suggesting a non-monotonic interaction between compression artifacts and learned forgery cues.

Under Gaussian noise, raw accuracy remains stable (66.67–67.96%) while Macro-F1 collapses to 0.400 even at $\text{std}=5$. This reveals that the classifier becomes biased toward the majority class (fake) under corruption, inflating accuracy while F1 deteriorates.

TABLE III. ROBUSTNESS RESULTS (FRAME LEVEL)

PERTURBATION	PARAM	ACC.	F1	AUC
None (baseline)	-	64.63%	0.604	0.637
JPEG	$q=80$	60.37%	0.589	0.642
JPEG	$q=60$	51.85%	0.515	0.583
JPEG	$q=40$	49.81%	0.498	0.569
JPEG	$q=20$	55.93%	0.553	0.596
Gauss.	$s=5$	67.96%	0.441	0.612
Gauss.	$s=10$	66.67%	0.400	0.503
Gauss.	$s=20$	66.67%	0.400	0.422
Gauss.	$s=40$	66.67%	0.400	0.440

D. Single-Image Inference

The final model supports local single-image inference. Using threshold $\tau = 0.168$, one example image produced $\text{prob real} = 0.6398$ and $\text{prob fake} = 0.3602$, yielding a prediction of FAKE. At the default threshold of 0.5 the same image is misclassified as REAL, illustrating the direct practical impact of threshold calibration.

V. PLANNED WORK AND TIMELINE

A. Remaining Work

(1) Finalise robustness plots and deepen interpretation of the Accuracy–Macro-F1 divergence under corruption. (2) Analyse threshold sensitivity under perturbation conditions. (3) Optionally extend the binary detector to manipulation-type classification across the four forgery families.

B. Risks and Mitigation

Overfitting is mitigated by the simple baseline and robustness experiments. Threshold sensitivity is mitigated by validation-based tuning. Data leakage is prevented by exclusive use of the official pair-level split files. CPU runtime constraints are addressed by MobileNetV2 and limited frame sampling.

C. Timeline

Short term: finalise robustness plots and discussion. Next phase: complete final report and presentation. Optional: multi-class manipulation-type extension.

VI. TEAM CONTRIBUTIONS

Abdelrahman Ibrahim Kamal Eldin

(Model Training & Tuning): Implemented the training pipeline, integrated the official pair-based split, trained and tuned the MobileNetV2 baseline across both stages, and ran all clean and robustness experiments. *Future:* Lead final model refinement, ablation study, and optional multi-class extension.

Esraa Mohammed Elsayed

(Data & Labelling Pipeline): Led dataset preparation and frame extraction, implemented Haar cascade face detection with fallback, verified the split protocol, and addressed class imbalance using WeightedRandomSampler. *Future:* Extend the data pipeline to support multi-class manipulation types.

Aya Said Abdallah Noah

(Robustness Testing & Reporting): Led robustness evaluation under JPEG and Gaussian noise, analysed frame-level and group-level results, interpreted class-bias effects, and prepared the midterm report and result figures. *Future:* Write the robustness analysis section, produce final figures, and coordinate presentation slides.

VII. USE OF GENAI TOOLS

In line with the course requirement for honest disclosure of generative AI use, the team used GenAI assistants during this project for the following purposes only.

Tools used: ChatGPT, Claude, and GitHub Copilot.

Code help and debugging: The tools were used to identify the cause of runtime errors and to suggest fixes for issues in the data extraction and training loops, including the Windows-related `num_workers = 0` setting in the DataLoader and the corrected Stage 2 learning rate. GitHub Copilot was also used for autocomplete of short, routine code patterns inside the notebook.

Explaining concepts: The tools were consulted to clarify our understanding of technical concepts used in the project, including the FaceForensics++ pair-based split protocol, the rationale for combining WeightedRandomSampler with class-weighted cross-entropy loss, validation-based threshold tuning, and the distinction between frame-level and group-level evaluation.

Where the tools were not used: GenAI tools were not used to generate experimental results, accuracy numbers, AUC values, or any figures in this report; all metrics in Tables I and II were produced by running the team’s own code on FaceForensics++ on local hardware. The tools were not used to write the report end-to-end, nor to fabricate references, datasets, or experimental conditions.

All GenAI suggestions were reviewed by the team before being incorporated into the code or the report, and the team is responsible for the correctness of all content in this submission.

REFERENCES

- [1] A. Rossler et al., “FaceForensics++: Learning to Detect Manipulated Facial Images,” in Proc. IEEE/CVF ICCV, 2019.
- [2] M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in Proc. IEEE/CVF CVPR, 2018.
- [3] F. Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,” in Proc. IEEE/CVF CVPR, 2017.
- [4] L. Li et al., “Face X-Ray for More General Face Forgery Detection,” in Proc. IEEE/CVF CVPR, 2020.
- [5] H. Qi et al., “DeepFake Detection by Analyzing Convolutional Traces,” IEEE Trans. Image Process., vol. 29, pp. 10832–10845, 2020.